

Funzionalità reali di `PopupHelper`

Non è un semplice popup statico, ma un sistema completo:

✓ **Popup automatico al passaggio del mouse**

```
PopupHelper.AttachPopup(...)
```

- associa il popup ad un qualsiasi `Control` ;
 - si attiva con `MouseEnter` ;
 - si chiude con `MouseLeave` ;
 - supporta immagini;
 - supporta colori personalizzati.
-

✓ **Popup manuale**

Hai anche:

```
PopupHelper.ShowPopup(...)
```

quindi si può mostrare un popup da codice in qualsiasi momento.

✓ **Gestione automatica posizione**

```
PopupForm.ShowNearControl
```

gestisce:

- apertura vicino al controllo;
 - controllo dei bordi dello schermo;
 - spostamento automatico sopra il controllo se non c'è spazio sotto.
-

✓ **Effetto Fade**

Il popup non appare improvvisamente:

```
Opacity = 0
```

poi:

```
FadeIn()
```

con incremento progressivo.

✓ Chiusura sicura

La gestione:

```
PopupHelper.HidePopup()
```

evita popup sovrapposti.

WinItaPascal – Popup e Utility

Introduzione

La libreria **WinItaPascal** mette a disposizione un sistema semplice e riutilizzabile per creare popup informativi nelle applicazioni **VB.NET WinForms**.

Il modulo permette di associare finestre popup personalizzate ai controlli dell'interfaccia utente senza dover creare manualmente form aggiuntivi.

Le caratteristiche principali sono:

- popup automatici al passaggio del mouse;
 - popup manuali;
 - immagini personalizzate;
 - colori configurabili;
 - effetto Fade In;
 - posizionamento automatico vicino al controllo.
-

Classe principale

La gestione dei popup viene effettuata tramite:

```
PopupHelper
```

La classe contiene i principali metodi:

Metodo	Descrizione
AttachPopup	Associa un popup ad un controllo
ShowPopup	Visualizza manualmente un popup
HidePopup	Chiude il popup attivo

AttachPopup

Descrizione

AttachPopup permette di associare un popup informativo ad un controllo WinForms.

Il popup viene mostrato automaticamente quando il mouse entra nel controllo.

Sintassi:

```
PopupHelper.AttachPopup(  
    ctrl,  
    message,  
    img,  
    backgroundColor,  
    textColor)
```

Parametri:

Parametro	Descrizione
ctrl	Controllo WinForms a cui associare il popup
message	Testo visualizzato
img	Immagine opzionale
backgroundColor	Colore sfondo popup
textColor	Colore testo popup

Esempio base

Esempio utilizzato nel form `FrmOrdini` :

```
Dim imgBeni As Image =  
    My.Resources.chegue  
  
PopupHelper.AttachPopup(  
    RjCircSalvaBeni,  
    vbCrLf &  
    "ATTENZIONE" &  
    vbCrLf &  
    vbCrLf &  
    "Salva eventuali modifiche",  
    imgBeni)
```

Quando l'utente posiziona il mouse sul pulsante viene visualizzato il popup.

Utilizzo con immagini

Le immagini possono essere caricate direttamente dalle risorse del progetto:

```
Dim imgInfo As Image =  
    My.Resources.cashier  
  
Dim imgWarning As Image =  
    My.Resources.calculator_50
```

Esempio:

```
PopupHelper.AttachPopup(  
    Button1,  
    "Informazioni utili" &  
    vbCr &  
    "Altre informazioni",  
    imgInfo)
```

```
PopupHelper.AttachPopup(  

```

```
Button2,  
"Attenzione! Controlla i dati",  
imgWarning)
```

Personalizzazione colori

È possibile personalizzare:

- colore dello sfondo;
- colore del testo.

Esempio:

```
Dim imgSalva As Image =  
    My.Resources.chegue  
  
PopupHelper.AttachPopup(  
    RjBtnSalva,  
    "Salva tutto",  
    imgSalva,  
    Color.AliceBlue,  
    Color.Blue)
```

Risultato:

```
Sfondo popup = AliceBlue  
Testo popup  = Blue
```

Colori predefiniti

Se i colori non vengono indicati, il framework utilizza automaticamente:

```
Sfondo = Yellow  
Testo  = Black
```

Esempio:

```
PopupHelper.AttachPopup(  
    RjBtnSalva,  
    "Salva tutto",  
    imgSalva)
```

ShowPopup

È possibile visualizzare manualmente un popup.

Esempio:

```
PopupHelper.ShowPopup(  
    RjBtnSalva,  
    "Operazione completata",  
    My.Resources.chegue)
```

Il popup viene mostrato immediatamente vicino al controllo indicato.

HidePopup

Chiude il popup attualmente visualizzato.

Esempio:

```
PopupHelper.HidePopup()
```

PopupForm

La classe interna utilizzata per la visualizzazione è:

```
PopupForm
```

Caratteristiche:

- finestra senza bordi;
- sempre in primo piano;

- ridimensionamento automatico;
- supporto immagini;
- layout dinamico;
- effetto Fade.

Posizionamento automatico

Il popup viene automaticamente posizionato vicino al controllo.

La libreria controlla:

- limite destro dello schermo;
- limite inferiore dello schermo;
- posizione minima visibile.

Se non è possibile mostrarlo sotto il controllo, viene visualizzato sopra.

AutoClose

È disponibile anche la chiusura automatica:

```
PopupForm.AutoClose(3000)
```

dove il valore indica i millisecondi prima della chiusura.

Utilizzo consigliato

Associare i popup nel metodo `Load` del form:

```
Private Sub FrmMain_Load(  
    sender As Object,  
    e As EventArgs) Handles MyBase.Load  
  
    PopupHelper.AttachPopup(  
        BtnSalva,  
        "Salva documento",
```

My.Resources.save)

End Sub

Vantaggi

L'utilizzo di `PopupHelper` permette di:

- migliorare l'esperienza utente;
- aggiungere informazioni contestuali;
- evitare finestre informative separate;
- mantenere uniforme lo stile dell'applicazione.

Note tecniche

- Compatibile con controlli WinForms standard e personalizzati.
- Le immagini vengono normalmente caricate tramite `My.Resources`.
- La gestione degli errori utilizza `FrameworkLogger`.
- Il popup attivo viene gestito centralmente dalla libreria.



Video dedicato

WinItaIPascal #07 - Popup e Utility



Nell'esempio era una libreria dedicata, ora è incluso in WinItaIPascal

[Esempio PopUp](#)

Esempio in Win Spinnet

[Win PopUp in Win Spinnet](#)



Repository GitHub



Pacchetto NuGet

[Install da NuGet](#)



Documentazione

Consulta anche:

- README.md
 - README_Database.md
 - README_GridUtility.md
 - README_Reports.md
 - README_Forms.md
 - README_Logging.md
 - README_IPMessageBox
 - CHANGELOG.md
-

Versione documentazione: 2.0.6

WinItaIPascal – Popup e Utility